

ADR-007: ingest `.rar` as a container, not only as something to unpack

Status: accepted, 2026-11-01 **Extends ADR-003. Constrained by ADR-001.**

Context

`.rar` was the one archive shape the library met and could not read. It surfaced as a real gap rather than a theoretical one: two files sitting in a download folder waiting to be imported, `Marathon Hero 2.rar` (6.3 GB) and `Guitar Hero On Tour - Modern Hits.rar` (182 MB), neither of which the server would index and neither of which `unpack` would open.

ADR-003 took one dependency to read `.7z` and recorded what it cost. The same question had to be answered again for RAR, and one more besides — whether a Go RAR reader can support the *shape* the rest of the scanner needs.

That second question is not cosmetic. `archiveItem` carries `open func() (io.ReadCloser, error)`: a **lazy, repeatable, random-access** opener per entry, and `newArchiveFS` maps every entry up front. A sequential-only reader cannot satisfy that contract. RAR is also a format where solid compression is normal, and on a solid `.7z` this project measured reading one entry out of order at **about three minutes** — the cost that took the live server down for ten minutes on 2026-11-01 and had to be rolled back.

So the risk was a reader that technically works and is unaffordable, which is worse than no reader at all: it would index an archive the server then cannot serve from.

Decision

Ingest `.rar` via `github.com/nwaples/rardecode/v2`, as a first-class container alongside `.zip` and `.7z` — indexed by the library walk, servable, packable and unpackable, through exactly the same code path.

`rardecode` offers three ways in: a sequential `Reader`, an `fs.FS` (`OpenFS`), and `List` plus a per-entry `File.Open`. The third matches `archiveItem` with no adapter, which is why it is the one used.

What it actually costs, measured rather than estimated

	before	after
Third-party packages compiled into the server	281	282
`windows/amd64` binary	13,512,192 bytes	13,961,728 bytes

+1 package, +0.43 MB. Against sevenzip's +68 packages and +1.62 MB this is close to free, because `rardecode` is pure Go with no dependencies of its own. All six release targets still cross-compile with `CGO_ENABLED=0`.

Whether reaching into a `.rar` is affordable: measured, on the real files

`internal/scan/rarorder_probe_test.go` is the probe, kept because the numbers are the whole basis of this decision. Measured 2026-11-01:

	Modern Hits	Marathon Hero 2
Size	173 MB	5,976 MB
Entries	307	951
`List` (enumerate, decompress nothing)	1.098 s	**0.033 s**
`solid` entries	0	0
Encrypted entries	0	0
Names that are not valid UTF-8	0	0

And on the access-order question that decided the `.7z` design:

	Modern Hits
Open the **first** file entry	1 ms
Open the **last** file entry, out of order	**< 1 ms**
40 entries in stored order	498 ms
40 entries through the sequential reader	477 ms

There is no out-of-order penalty. Stored order and sequential order cost the same, and a late entry costs what the first one costs. That is the opposite of `.7z`, where stored order was 3m12s for 616 entries and out-of-order was about 3m07s *each*. RAR's seekable header chain is also why enumerating 6 GB takes 33 ms where `.7z` needs to read a metadata block.

This is a property of these two archives, not a guarantee about the format. RAR supports solid compression, and a solid `.rar` would cost precisely what a solid `.7z` costs. It needs no special case because the remedy already exists and is the same one — `unpack` it — but the measurement above should not be read as "RAR is always cheap".

Multi-volume sets: only the first part is an entry point

`rardecode` follows volumes by name, so given `X.part1.rar` it reads the whole set. The library walk, left alone, would then offer it `X.part2.rar` as well, and both outcomes are bad:

- it fails, and the operator gets one spurious error per part; or
- it finds part 1 beside it and **succeeds**, so every song in the set is indexed once per part. The chart hash collapses the duplicate songs, so the visible symptom is not a doubled library but something subtler — the index reporting the wrong archive as a song's source.

So `IsContainer` excludes continuation parts, and `IsSecondaryRarVolume` decides that by parsing the digits as a **number**. Padding is the writer's choice: `part1`, `part01` and `part001` all occur in the wild, and a text comparison against `"1"` loses whole sets. That is exactly what the negative control demonstrated — reintroducing the text comparison turns `TestFirstRarVolumeIsAnEntryPointWhateverThePadding` red on the padded variants only.

The older `X.rar` / `X.r00` / `X.r01` convention needs nothing: those parts do not end in `.rar`, so they were never offered. A test pins that, so nobody adds `.r00` later meaning well.

There is no committed `.rar` fixture, deliberately

`rardecode` decodes only, there is no RAR writer for Go, and 7-Zip cannot create RAR either. A fixture would have to come from a hand-rolled RAR5 encoder, and this repo's own rule rules that out: round-tripping against our own writer "*only proves our reader and writer agree with each other*". A fixture my encoder produced would be a test of my encoder.

So the coverage splits:

- **`internal/scan/rar_test.go`** — everything decidable from a name or a few header bytes: container classification, continuation parts, padding, the `.r00` convention, the RAR 1.5-4.x and RAR 5.0 signatures, a short file, a missing file, and a text file merely *named* `.rar` (which must not be reported as `ErrUnopenableRar`, because that message sends the operator looking for volumes of a set that does not exist). Runs everywhere.
- **`TestRarContainerAgainstARealArchive`** — gated on `YARG_RAR_LIVE`, and the only thing that proves the feature. Against Modern Hits it found **44 songs** in 307 entries and packed every one of them. It now extracts the archive as well, because the read-only version of it passed on a build that could not unpack a single song folder — see the directory-flag section below.

RAR marks a directory with a flag, not a trailing slash — and only the WRITER cared

Found on 2026-11-01, unpacking the two archives this ADR was written for. The first `.rar` through the new reader died on its first song folder:

```
unpack: scan: .../Guitar Hero On Tour - Modern Hits.unpacking/Guitar Hero On Tour -
Modern Hits/12 Stones - Adrenaline is already there; two parts claim the same file
```

Three things in that message are misleading at once. It is a **split-set** error, on an archive with exactly **one** part; and the name it accuses of being duplicated appears once.

The cause is a handover, not a decoder. `cleanEntryName` reads directory-ness off a trailing `/`, because that is what the other two readers deliver. Probed, rather than assumed:

Archive	Directory entries	...ending in `/`
`Guitar Hero On Tour - Modern Hits.rar`	45	**0**
`GH5 Hidden Songs.7z`	6	6

`archive/zip` and `sevenzip` both append the slash; `rardecode` reports `FileHeader.IsDir` and leaves the name alone. So every one of those 45 directories was read as a zero-byte **file**.

The reason this shipped is that reading repairs it and writing cannot. `archiveFS.record` promotes a path to a directory the moment a child names it as an ancestor, so the `fs.FS` came out correct, all 44 songs were found, every one packed, and the live test passed. `writeOne` opens with `O_EXCL` — deliberately, so that two parts of a set claiming one path is an error rather than a silent overwrite — so the zero-byte file lands on the directory its own children just created, and the unpack aborts.

Two things follow, and both are now in the code:

- The conversion belongs at the **reader boundary**, in `storedName`, so `cleanEntryName` stays the single place that decides what a directory is. Not in `extract`, which would leave the `fs.FS` quietly depending on a repair it should never have needed.
- **A test that only reads cannot see this.** `TestRarContainerAgainstARealArchive` asserted enumeration, scanning and packing, and passed on the broken build. It now also *extracts* and walks the result as a real directory tree. The ungated half, `TestStoredNameMarksDirectoriesTheWayCleanEntryNameReadsThem`, needs no fixture because the defect was never in RAR decoding: it checks both ends of the handover agree, and was watched going red against the old behaviour before being trusted.

The general shape is worth keeping: **a forgiving reader hides a strict writer.** Any new container added here is exercised through extraction, not only through the `fs.FS`.

Consequences

Good

- The two archives that prompted this are now ordinary library content.
- `.rar` reaches the unpack machinery for free, including `expandNested`, so a `.rar` inside an archive is opened like any other container.
- `worthUnpacking` includes it, so `Archives()` lists it and the admin page offers it.
- The upload check accepts `.rar`, so a client is told what the server will do with one.

Bad, or at least owed

- A **solid** `.rar` will be as slow as a solid `.7z`, and nothing warns about it in advance. `FileHeader.Solid` is available per entry, so a warning is cheap to add once somebody meets one; it is not added now because no archive here is solid and an untested warning path is not an improvement.
- An encrypted `.rar` is refused with `ErrUnopenableRar`. The message leads with the missing- volume case rather than the password case, because that is the likelier cause for RAR and because a missing volume is something the operator can act on.
- No committed fixture, as above, so a change to the RAR path can go green on CI and still be wrong. The live test is the guard and it has to be run deliberately.

What is still not ingested

Nothing changes in ADR-003's refusals. `.con`, `_rb3con`, `.pkg` and `.xex` remain recognised and refused with a stated reason; CON/mogg decryption remains a permanent non-goal.